Run in Google Colab View on Github View raw on Github

Module 7: 1D data

Let's first import basic packages and then load a dataset from `vega_datasets` package. If you don't have `vega_datasets` or `altair` installed yet, use `pip` or `conda` to install them.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline

from vega_datasets import data


cars = data.cars()
cars.head()
```

	Name	Miles_per_Gallon	Cylinders	Displacement	Horsepower	Weight_in_lb
0	chevrolet chevelle malibu	18.0	8	307.0	130.0	350
1	buick skylark 320	15.0	8	350.0	165.0	369
2	plymouth satellite	18.0	8	318.0	150.0	343

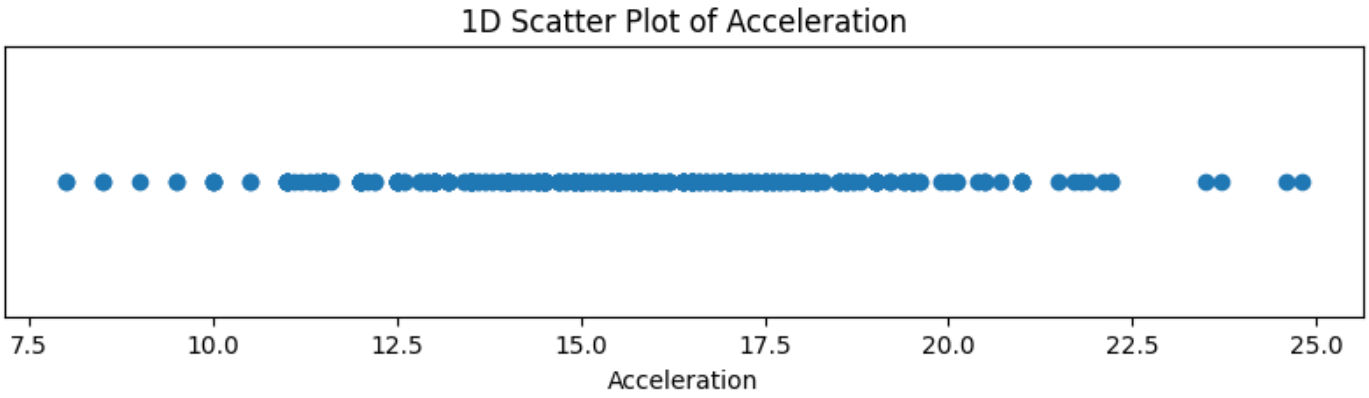
Next steps: Generate code with `cars` View recommended plots New interactive sheet

1D scatter plot

Let's consider the `Acceleration` column as our 1D data. If we ask pandas to plot this series, it'll produce a line graph where the index becomes the horizontal axis.

```
# Load the dataset
cars = data.cars()

# Create the 1D scatter plot
plt.figure(figsize=(10, 2))
plt.scatter(cars.Acceleration, np.zeros_like(cars.Acceleration))
plt.yticks([])  # Remove y-axis ticks
plt.xlabel("Acceleration")
plt.title("1D Scatter Plot of Acceleration")
plt.show()
```



Because the index is not really meaningful, drawing line between subsequent values is misleading! This is definitely not the plot we want!

It's actually not trivial to use pandas to create an 1-D scatter plot. Instead, we can use `matploblib`'s `scatter` function. We can first create an array with zeros that we can use as the vertical coordinates of the points that we will plot. `np.zeros_like` returns an array with zeros that matches the shape of the input array.

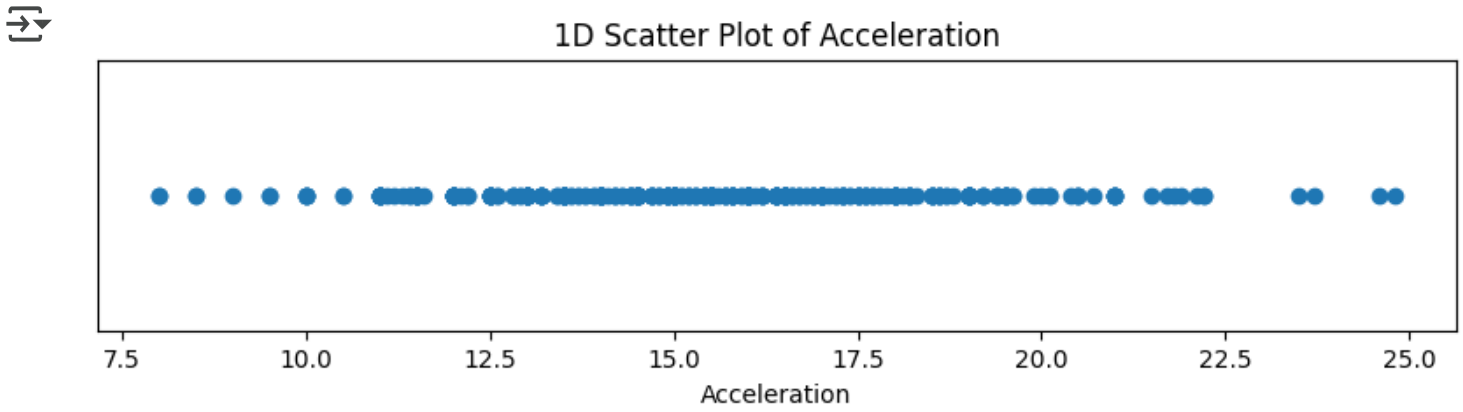
```
np.zeros_like([1,2,3])
```

```
array([0, 0, 0])
```

Q: now can you create an 1D scatter plot wit `matplotlib`'s scatter function? Make the figure wide (e.g. set `figsize=(10,2)`) and then remove the y tics.

```
# YOUR SOLUTION HERE

plt.figure(figsize=(10, 2))
plt.scatter(cars.Acceleration, np.zeros_like(cars.Acceleration))
plt.yticks([])  # Remove y-axis ticks
plt.xlabel("Acceleration")
plt.title("1D Scatter Plot of Acceleration")
plt.show()
```



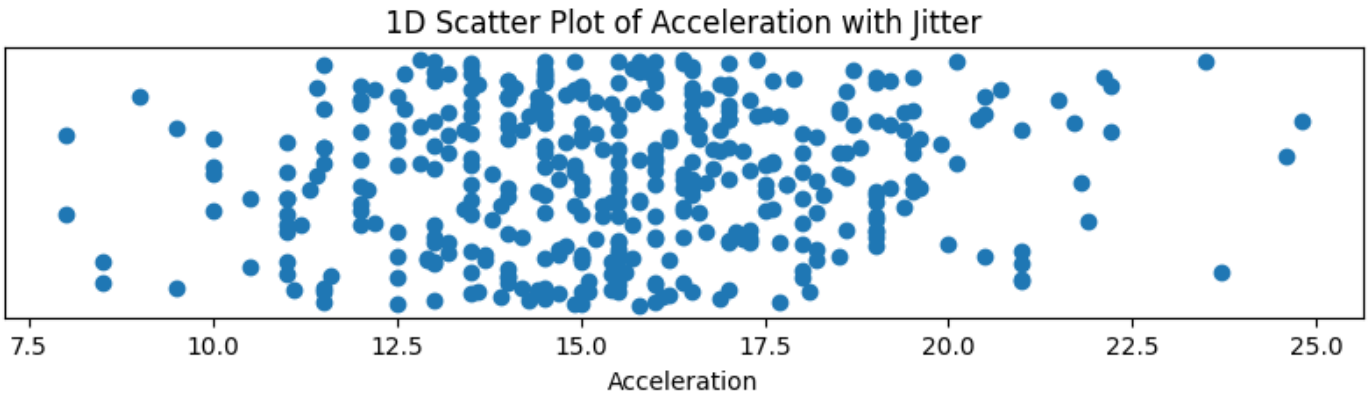
As you can see, there are lots of occlusions. So this plot cannot show the distribution properly and we would like to fix it. How about adding some jitters? You can use `numpy`'s `random.rand()` function to generate random numbers, instead of using an array with zeros.

Q: create a jittered 1D scatter plot.

```
# jittered_y = ...

# YOUR SOLUTION HERE

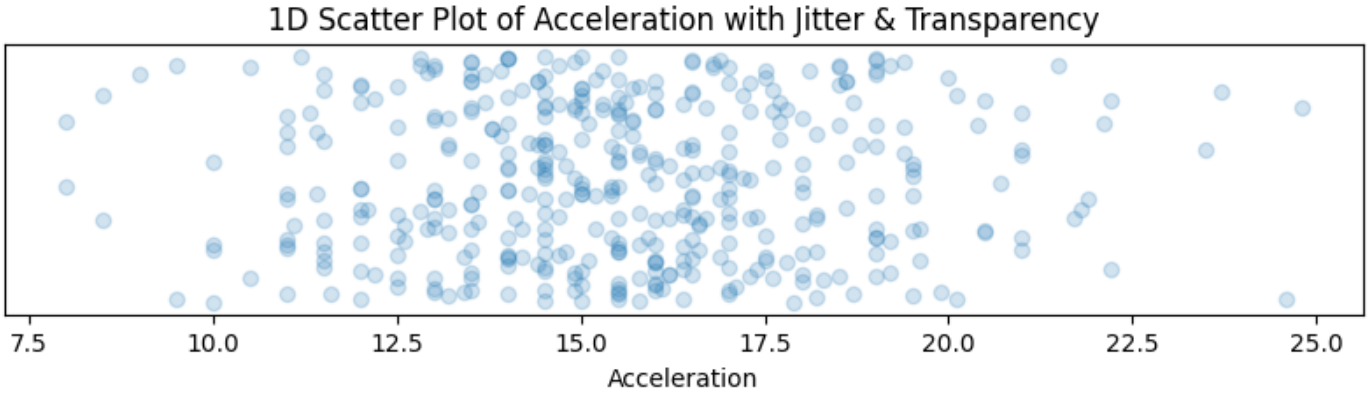
plt.figure(figsize=(10, 2))
jitter = np.random.rand(len(cars.Acceleration)) * 0.3  # Small random values
plt.scatter(cars.Acceleration, jitter)
plt.yticks([])  # Hide y-axis ticks
plt.xlabel("Acceleration")
plt.title("1D Scatter Plot of Acceleration with Jitter")
plt.show()
```



We can further improve this by adding transparency to the symbols. The transparency option for `scatter` function is called `alpha`. Set it to be 0.2.

Q: create a jittered 1D scatter plot with transparency (alpha=0.2)

```
# YOUR SOLUTION HERE
plt.figure(figsize=(10, 2))
jitter = np.random.rand(len(cars.Acceleration)) * 0.3
plt.scatter(cars.Acceleration, jitter, alpha=0.2)  # Add transparency
plt.yticks([])  # Hide y-axis ticks
plt.xlabel("Acceleration")
plt.title("1D Scatter Plot of Acceleration with Jitter & Transparency")
plt.show()
```

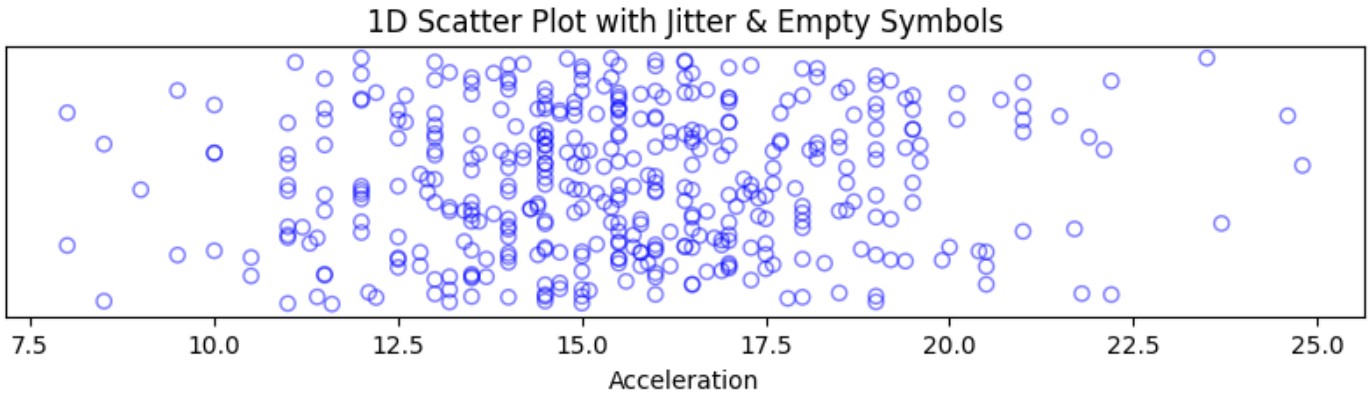


Another strategy is using empty symbols. The option is `facecolors`. You can also change the stroke color (`edgecolors`).

Q: create a jittered 1D scatter plot with empty symbols.

```
# YOUR SOLUTION HERE

plt.figure(figsize=(10, 2))
jitter = np.random.rand(len(cars.Acceleration)) * 0.3
plt.scatter(
    cars.Acceleration, jitter,
    facecolors='none', edgecolors='blue', alpha=0.5
)
plt.yticks([])
plt.xlabel("Acceleration")
plt.title("1D Scatter Plot with Jitter & Empty Symbols")
plt.show()
```



What happens if you have lots and lots of points?

Whatever strategy that you use, it's almost useless if you have too many data points. Let's play with different number of data points and see how it looks.

It not only becomes completely useless, it also take a while to draw the plot itself.

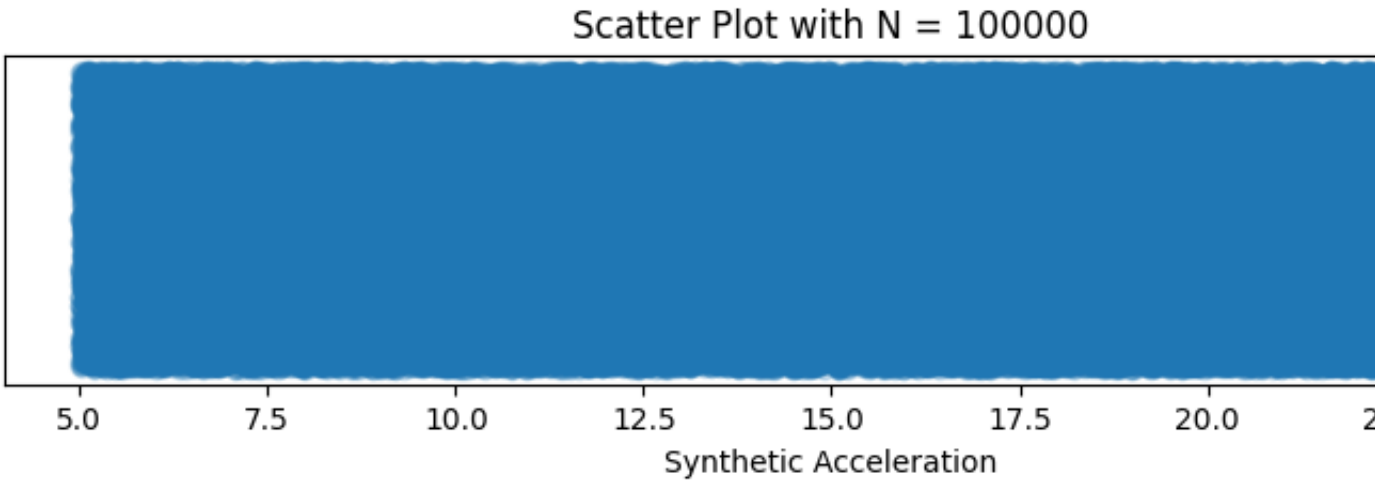
```
# TODO: play with N and see what happens.
N = 100000
x = np.random.rand(N)
jittered_y = np.random.rand(N)

# YOUR SOLUTION HERE

import numpy as np
import matplotlib.pyplot as plt

# Generate synthetic data
N = 100000
x = np.random.rand(N) * 20 + 5   # Spread from 5 to 25 (like Acceleration)
jittered_y = np.random.rand(N) * 0.3  # Small vertical jitter

# Plot
plt.figure(figsize=(10, 2))
plt.scatter(x, jittered_y, alpha=0.2)  # Add transparency
plt.yticks([])  # Hide y-axis ticks
plt.xlabel("Synthetic Acceleration")
plt.title(f"Scatter Plot with N = {N}")
plt.show()
```



Histogram and boxplot

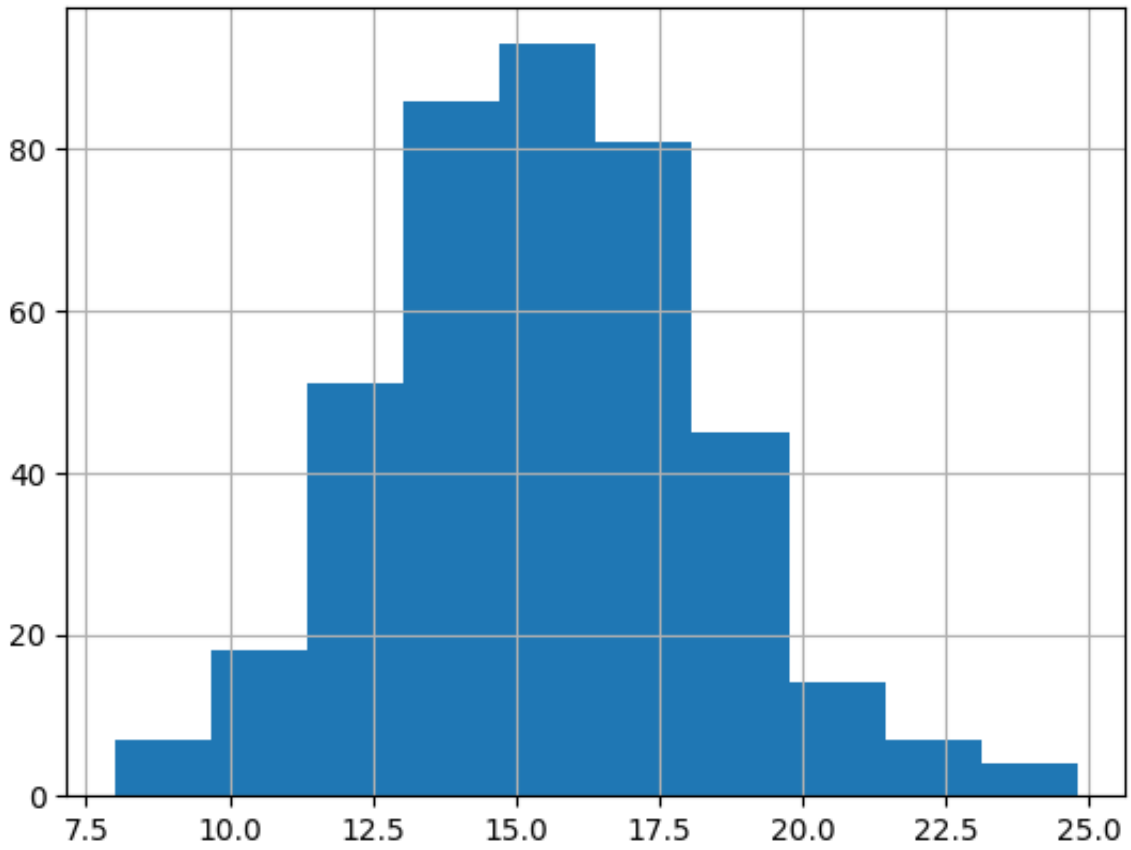
When you have lots of data points, you can't no longer use the scatter plots. Even when you don't have millions of data points, you often want to get a quick summary of the distribution rather than seeing the whole dataset. For 1-D datasets, two major approaches are histogram and boxplot. Histogram is about aggregating and counting the data while boxplot is about summarizing the data. Let's first draw some histograms.

Histogram

It's very easy to draw a histogram with pandas.

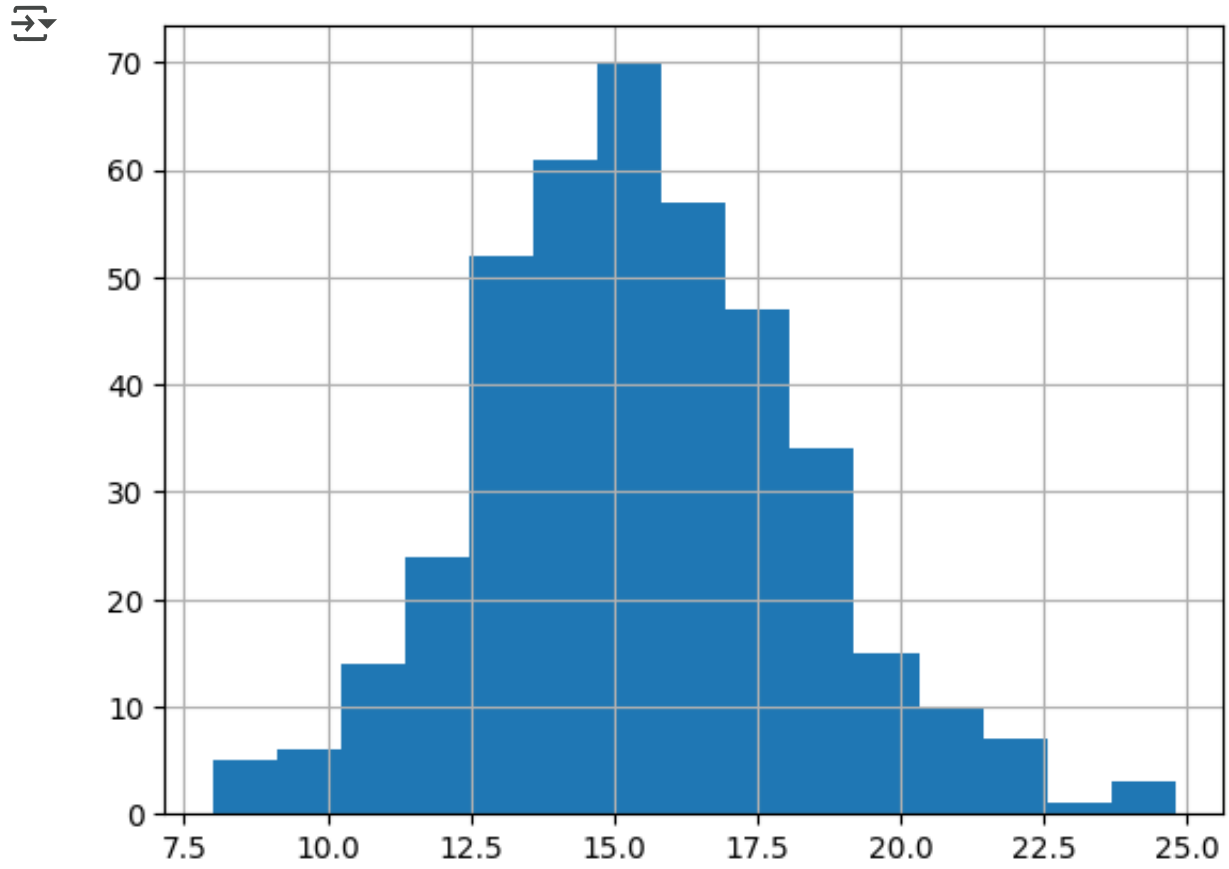
```
cars.Acceleration.hist()
```

```
<Axes: >
```



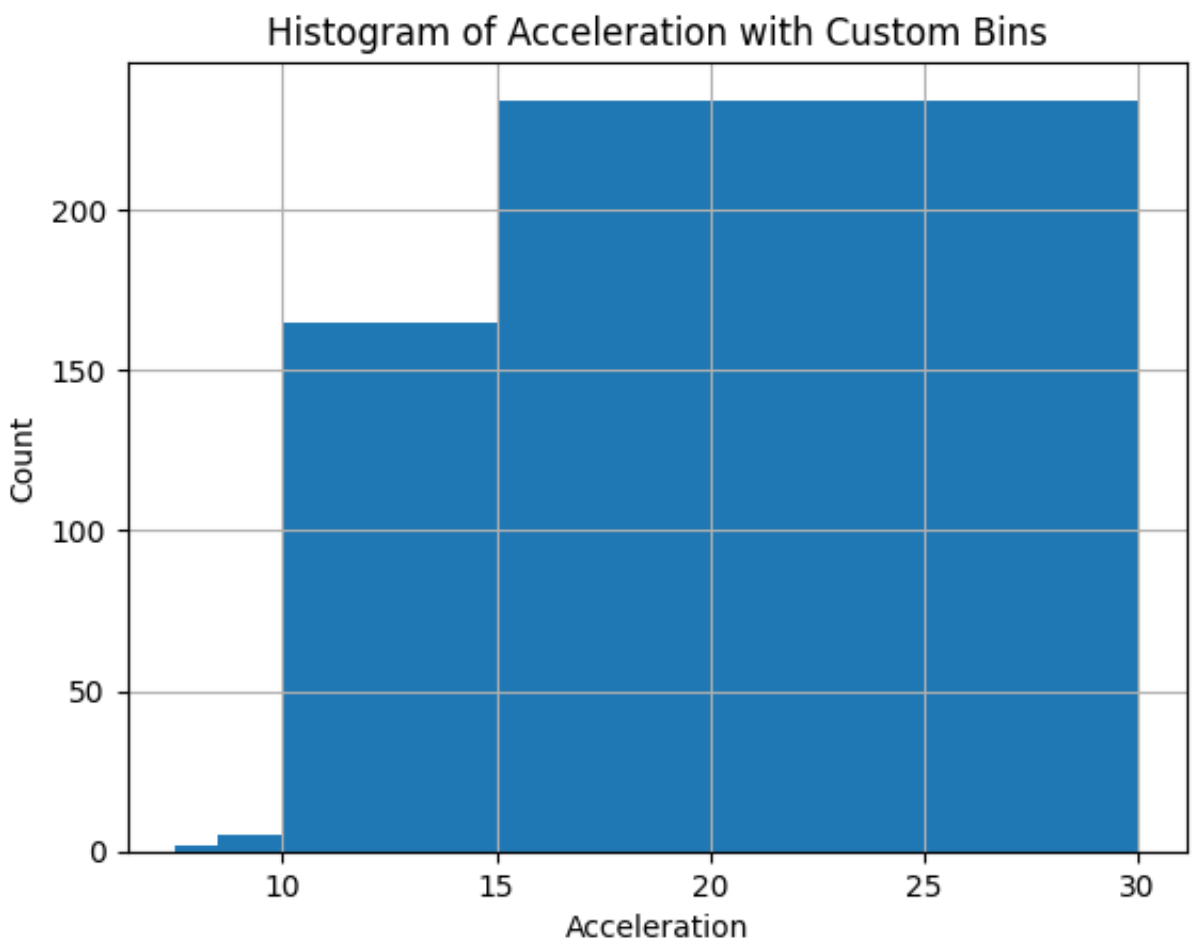
You can adjust the bin size, which is the main parameter of the histogram.

```
cars.Acceleration.hist(bins=15)
bins = [7.5, 8.5, 10, 15, 30]
```



You can even specify the actual bins.

```
bins = [7.5, 8.5, 10, 15, 30]
cars.Acceleration.hist(bins=bins)
plt.xlabel("Acceleration")
plt.ylabel("Count")
plt.title("Histogram of Acceleration with Custom Bins")
plt.show()
```

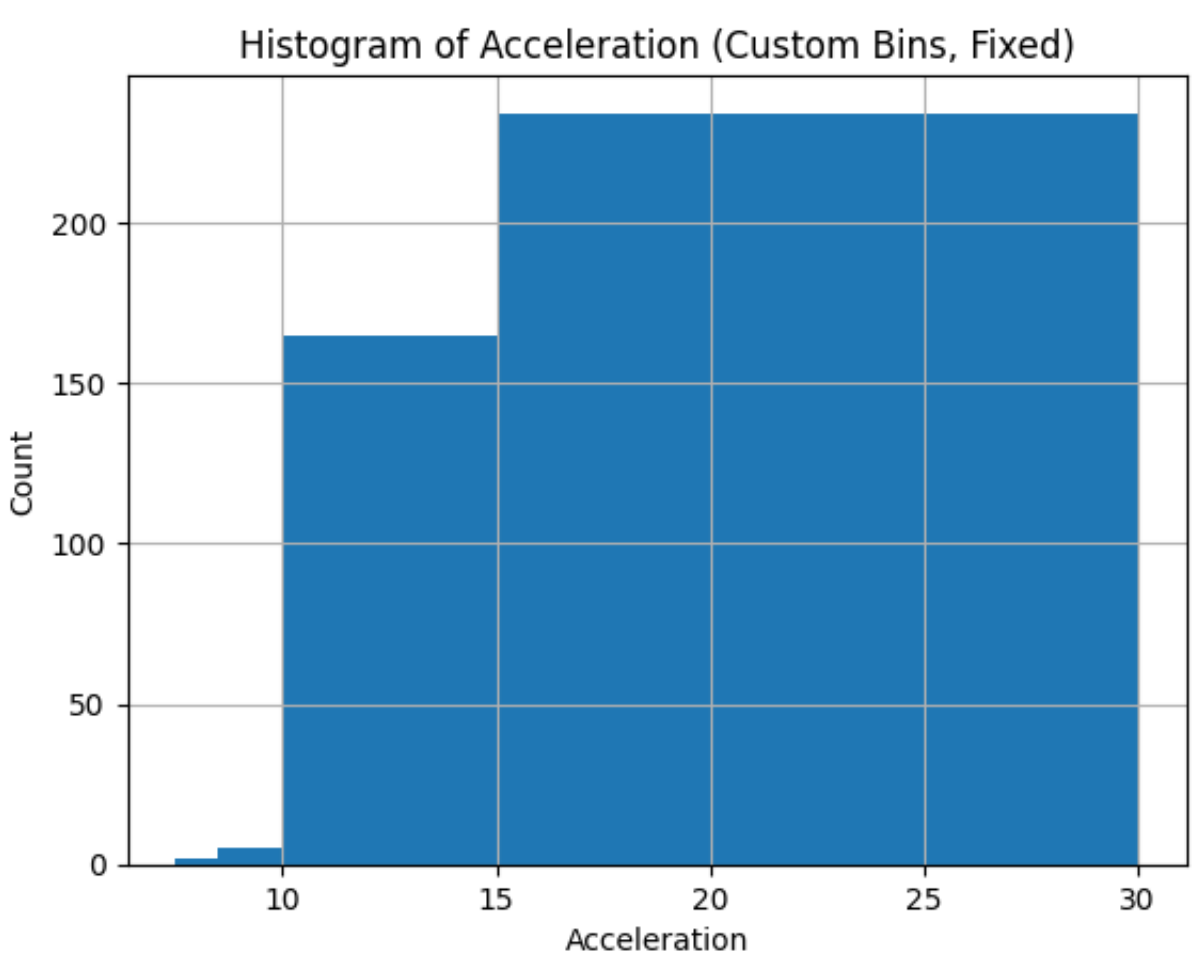


Do you see anything funky going on with this histogram? What's wrong? Can you fix it?

Q: Explain what's wrong with this histogram and fix it.

(hints: do you remember what we discussed regarding histogram? Also pandas documentation does not show the option that you should use. You should take a look at the `matplotlib`'s documentation.

```
# YOUR SOLUTION HERE
bins = [7.5, 8.5, 10, 15, 30]
cars.Acceleration.hist(bins=bins, density=False)  # Force count mode
plt.xlabel("Acceleration")
plt.ylabel("Count")
plt.title("Histogram of Acceleration (Custom Bins, Fixed)")
plt.show()
```



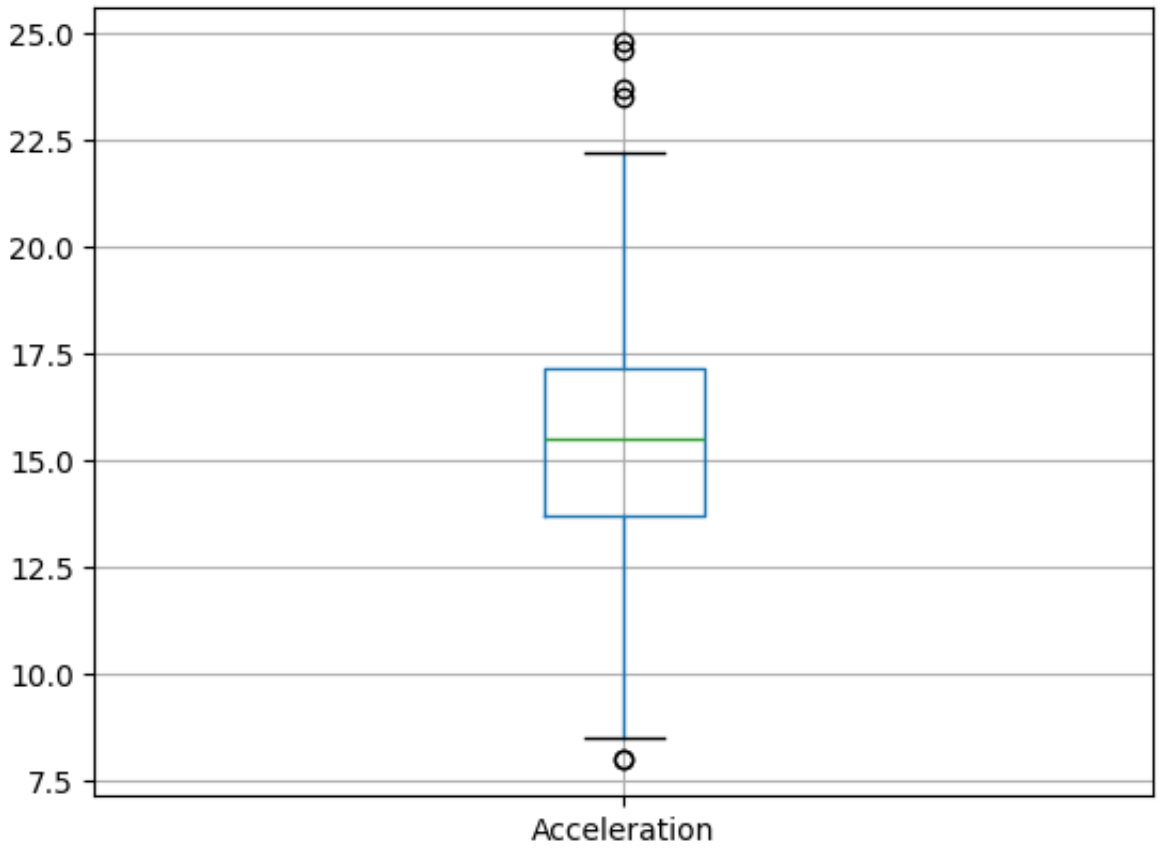
Boxplot

Boxplot can be created with pandas very easily. Check out the `plot` documentation:
https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.plot.html

Q: create a box plot of `Acceleration`

```
# YOUR SOLUTION HERE
cars.boxplot(column="Acceleration")
```

```
<Axes: >
```



Comparison of Visualization Methods for `Acceleration`

The three plots each give a different perspective on the `Acceleration` data:

- **1D scatter plot with jitter** displays each individual data point. It's helpful for visualizing the raw distribution and spotting clusters or gaps. However, it can become hard to interpret when there's too much data or overplotting — even with jitter and transparency.
- **Histogram** aggregates the data into bins, which makes it much easier to see the general shape and density of the distribution. It shows which ranges have more or fewer data points. The downside is that bin size can affect how the data looks, so interpretation depends on that choice.
- **Boxplot** summarizes the distribution using five statistics: minimum, Q1, median, Q3, and maximum. It’s ideal for spotting outliers and understanding spread and skewness quickly. However, it hides the exact distribution details that the histogram or scatter plot reveals.

Conclusion:
Each plot has a purpose — scatter plots are great for raw data inspection, histograms for distribution shape, and boxplots for statistical summary and outlier detection. Together, they provide a complete picture of the `Acceleration` variable.

1D scatter plot with Seaborn and Altair

As you may have noticed, it is not very easy to use `matplotlib`. The organization of plot functions and parameters are not very systematic. Whenever you draw something, you should search how to do it, what are the parameters you can tweak, etc. You need to manually tweak a lot of things when you work with `matplotlib`.

There are more systematic approaches towards data visualization, such as the "Grammar of Graphics". This idea of *grammar* led to the famous `ggplot2` (http://ggplot2.tidyverse.org) package in R as well as the Vega & Vega-lite for the web. The grammar-based approach lets you work with *tidy data* in a natural way, and also lets you approach the data visualization systematically. In other words, they are very cool. 😎

I'd like to introduce two nice Python libraries. One is called `seaborn` (https://seaborn.pydata.org), which is focused on creating complex statistical data visualizations, and the other is called `altair` (https://altair-viz.github.io/) and it is a Python library that lets you *define* a visualization and translates it into vega-lite json.

Seaborn would be useful when you are doing exploratory data analysis; altair may be useful if you are thinking about creating and putting an interactive visualization on the web.

If you don't have them yet, check the installation page of altair. In `conda`,

```
$ conda install -c conda-forge altair vega_datasets jupyterlab
```

Let's play with it.

```
import seaborn as sns
import altair as alt


# Uncomment the following line if you are using Jupyter notebook
# alt.renderers.enable('notebook')
```

```
cars.head()
```

	Name	Miles_per_Gallon	Cylinders	Displacement	Horsepower	Weight_in_lb
0	chevrolet chevelle malibu	18.0	8	307.0	130.0	350
1	buick skylark 320	15.0	8	350.0	165.0	369
2	plymouth satellite	18.0	8	318.0	150.0	343

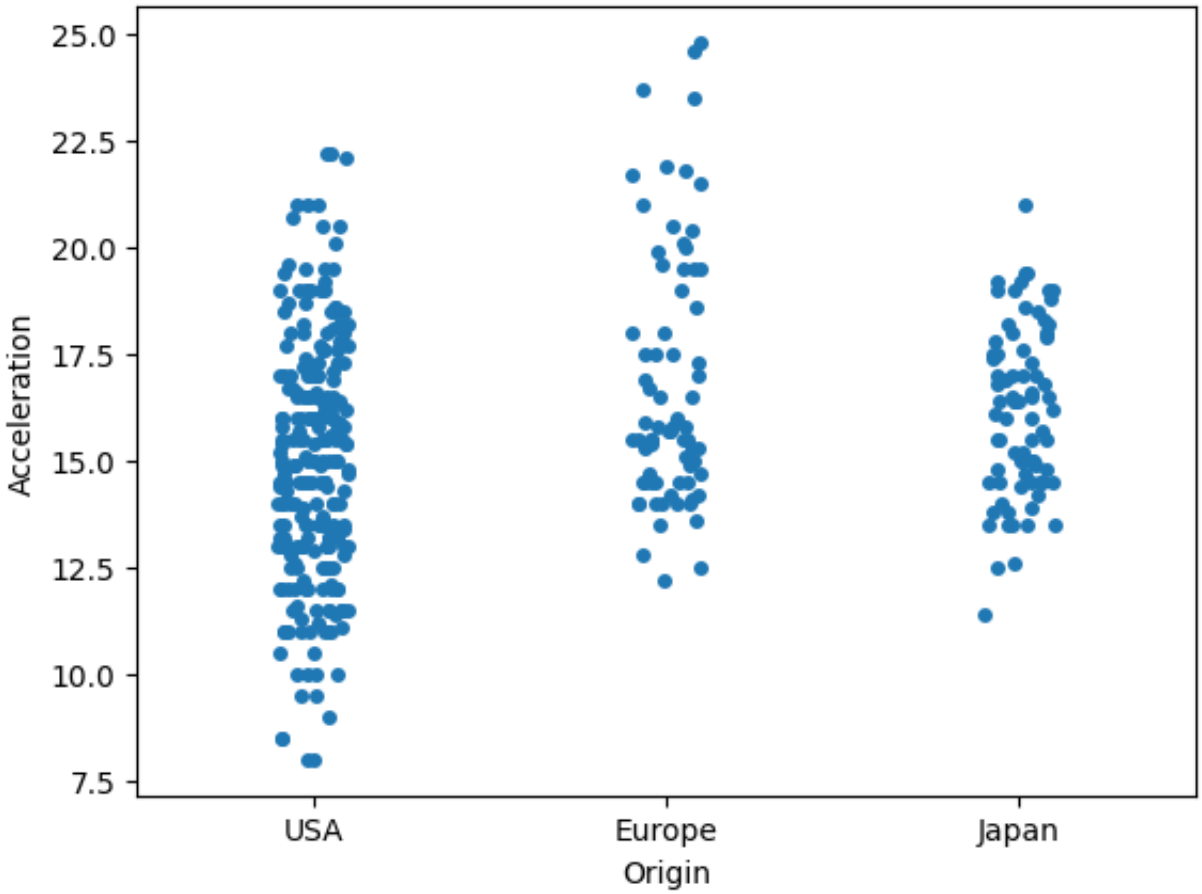
Next steps: Generate code with cars | View recommended plots | New interactive sheet

Beeswarm plots with seaborn

Seaborn has a built-in function to create 1D scatter plots with multiple categories.

```
sns.stripplot(x='Origin', y='Acceleration', data=cars)
```

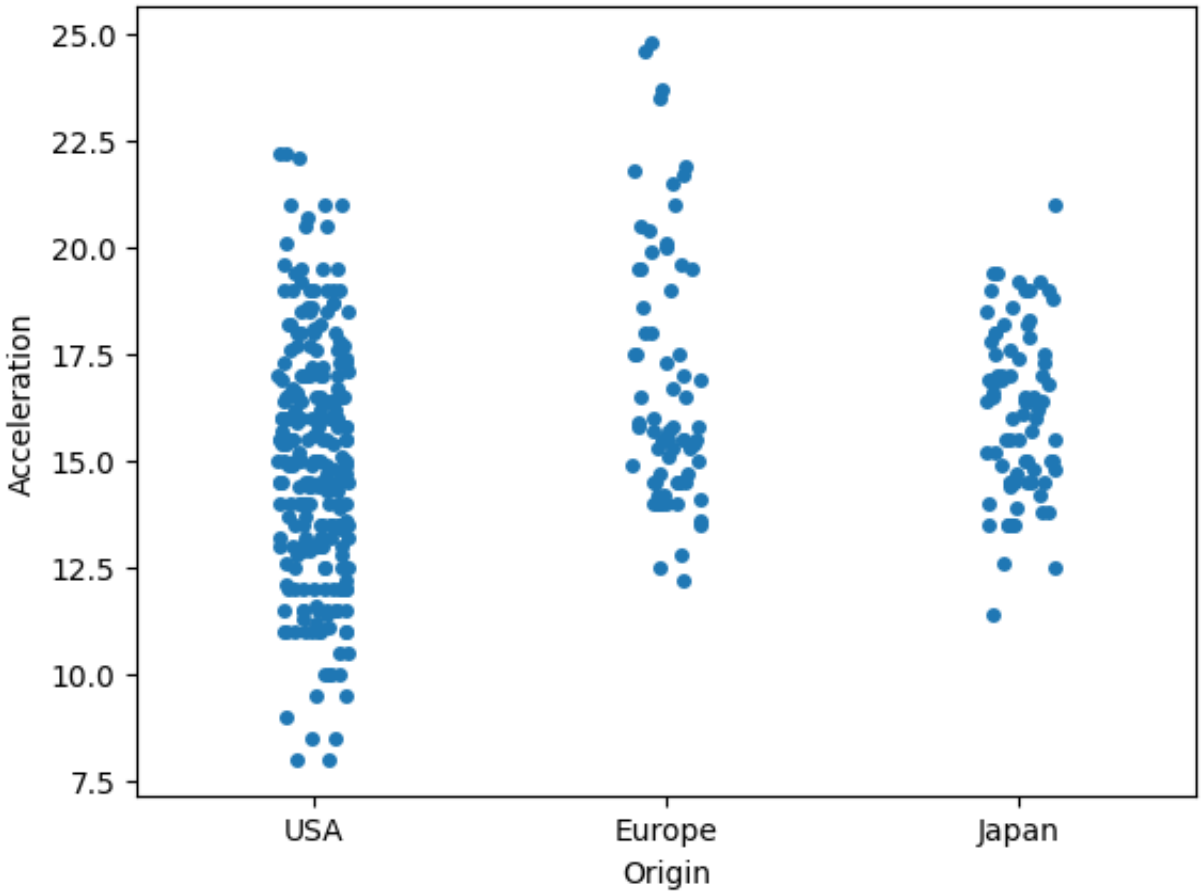
```
<Axes: xlabel='Origin', ylabel='Acceleration'>
```



And you can easily add jitters or even create a beeswarm plot.

```
sns.stripplot(x='Origin', y='Acceleration', data=cars, jitter=True)
```

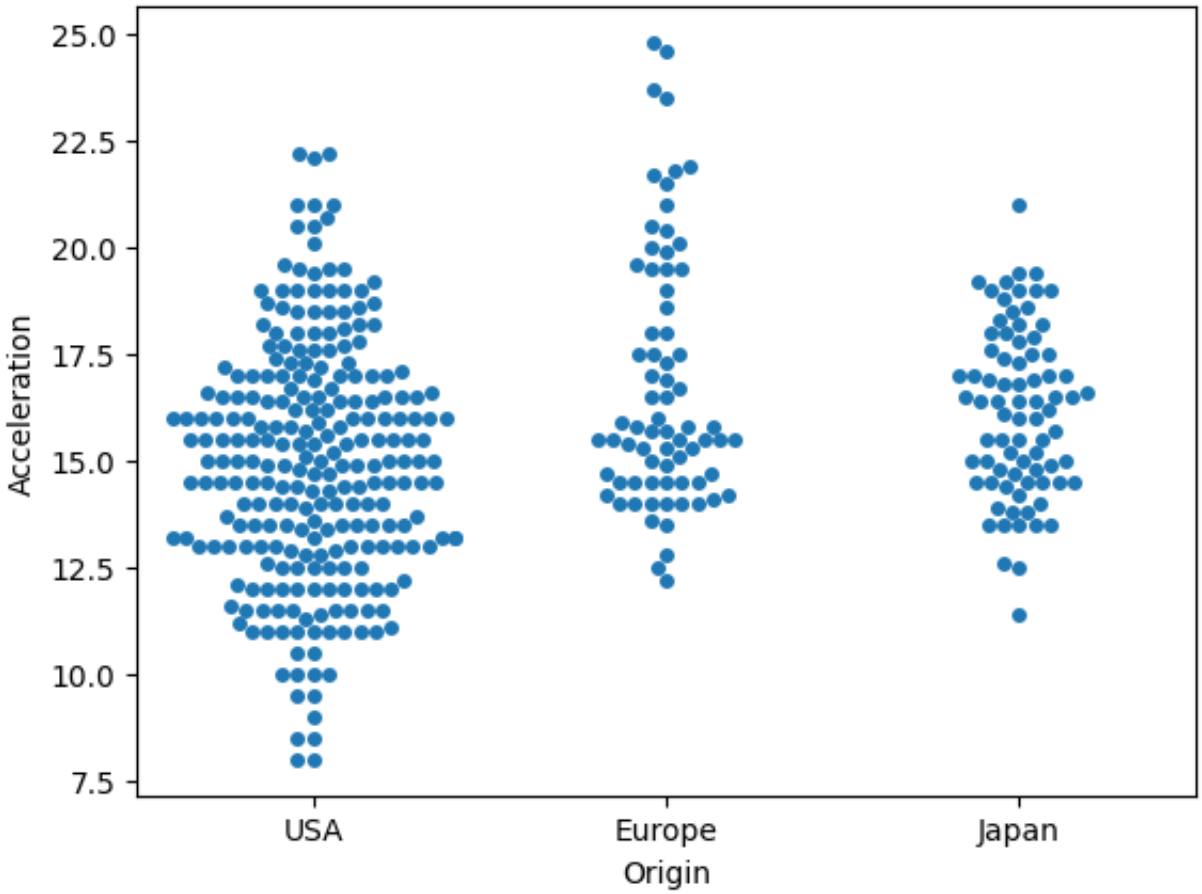
```
<Axes: xlabel='Origin', ylabel='Acceleration'>
```



Seems like European cars tend to have good acceleration. 😎 Let's look at the beeswarm plot, which is a pretty nice option for fairly small datasets.

```
sns.swarmplot(x='Origin', y='Acceleration', data=cars)
```

```
<Axes: xlabel='Origin', ylabel='Acceleration'>
```

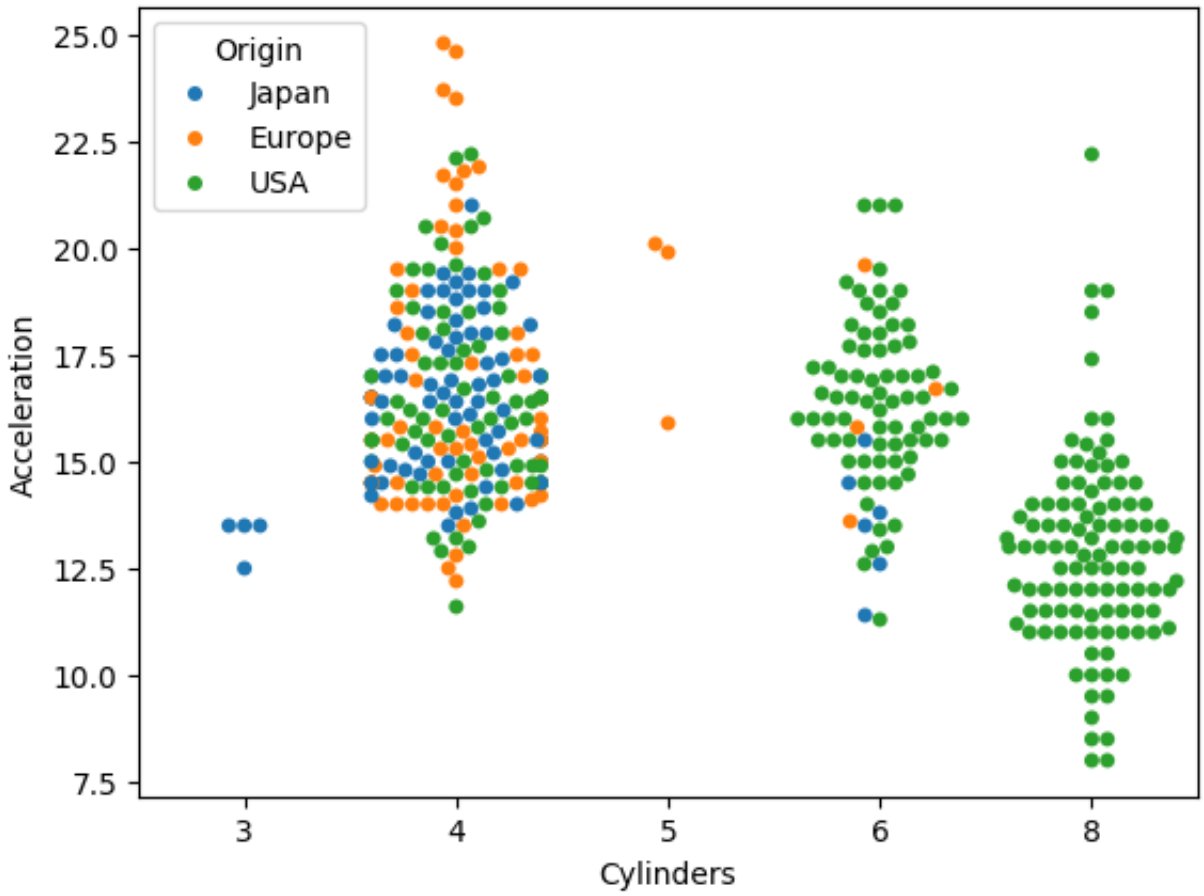


Seaborn also allows you to use colors for another categorical variable. The option is `hue`.

Q: can you create a beeswarm plot where the swarms are grouped by `Cylinders`, y-values are `Acceleration`, and colors represent the `Origin`?

```
# YOUR SOLUTION HERE
sns.swarmplot(x='Cylinders', y='Acceleration', hue='Origin', data=cars)
```

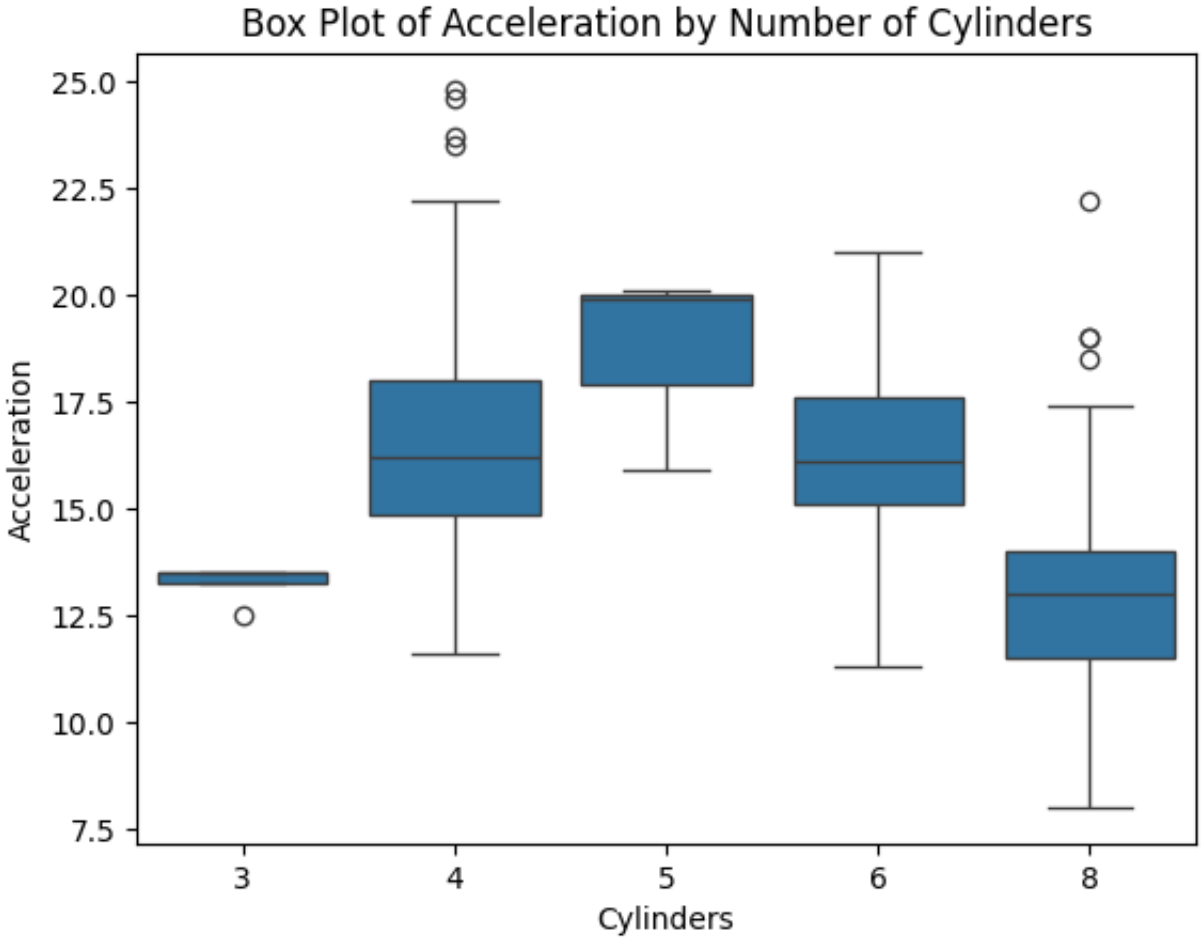
```
/usr/local/lib/python3.11/dist-packages/seaborn/categorical.py:3399: UserWarni
  warnings.warn(msg, UserWarning)
<Axes: xlabel='Cylinders', ylabel='Acceleration'>
/usr/local/lib/python3.11/dist-packages/seaborn/categorical.py:3399: UserWarni
  warnings.warn(msg, UserWarning)
```



And of course you can create box plots too.

Q: Create boxplots to show the relationships between `Cylinders` and `Acceleration`.

```
# YOUR SOLUTION HERE
sns.boxplot(x='Cylinders', y='Acceleration', data=cars)
plt.xlabel('Cylinders')
plt.ylabel('Acceleration')
plt.title('Box Plot of Acceleration by Number of Cylinders')
plt.show()
```



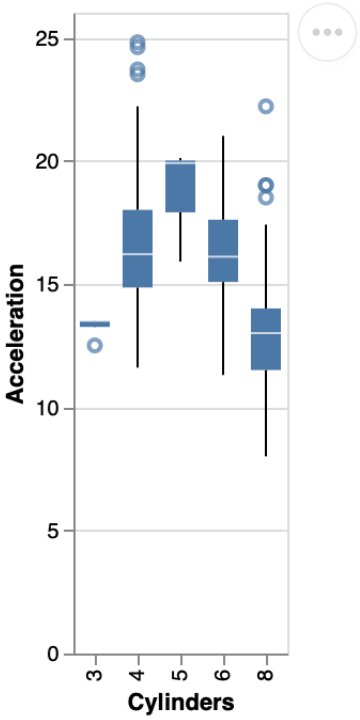
Altair basics

With `altair`, you're thinking in terms of a whole dataframe, rather than vectors for x or vectors for y. Passing the dataset to `Chart` creates an empty plot. If you try to run `alt.Chart(cars)`, it will complain. You need to say what's the visual encoding of the data.

```
import altair as alt

chart = (
    alt.Chart(cars)
       .mark_boxplot()
       .encode(
           x=alt.X('Cylinders:O', title='Cylinders'),
           y=alt.Y('Acceleration:Q', title='Acceleration')
       )
)

chart
```

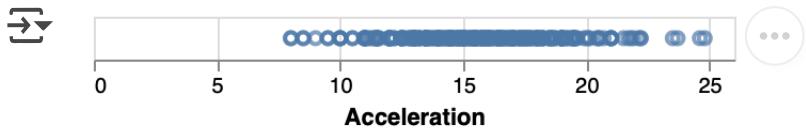



```
alt.Chart(cars).mark_point()
```



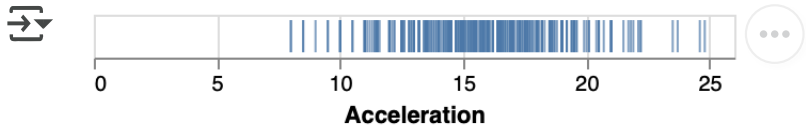
So you just see one *point*. But actually this is not a single point. This is every row of the dataset represented as a point at the same location. Because there is no specification about where to put the points, it simply draws everything on top of each other. Let's specify how to spread them across the horizontal axis.

```
alt.Chart(cars).mark_point().encode(
    x='Acceleration',
)
```



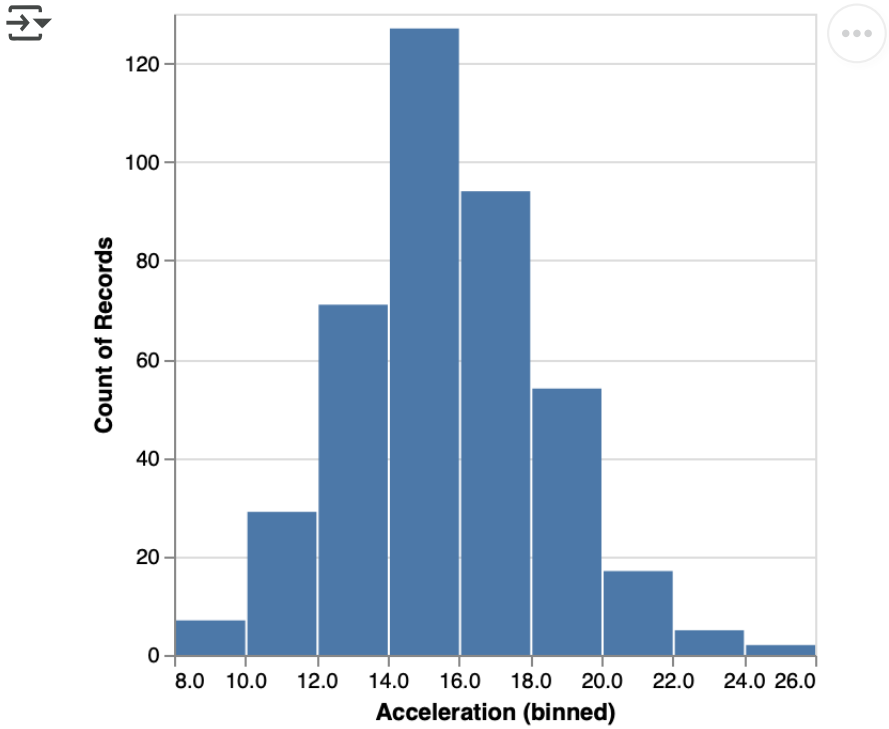
There is another nice mark called `tick`:

```
alt.Chart(cars).mark_tick().encode(
    x='Acceleration',
)
```



In `altair`, histogram is not a special type of visualization, but simply a plot with bars where a variable is binned and a counting aggregation function is used.

```
alt.Chart(cars).mark_bar().encode(
    x=alt.X('Acceleration', bin=True),
    y='count()'
)
```

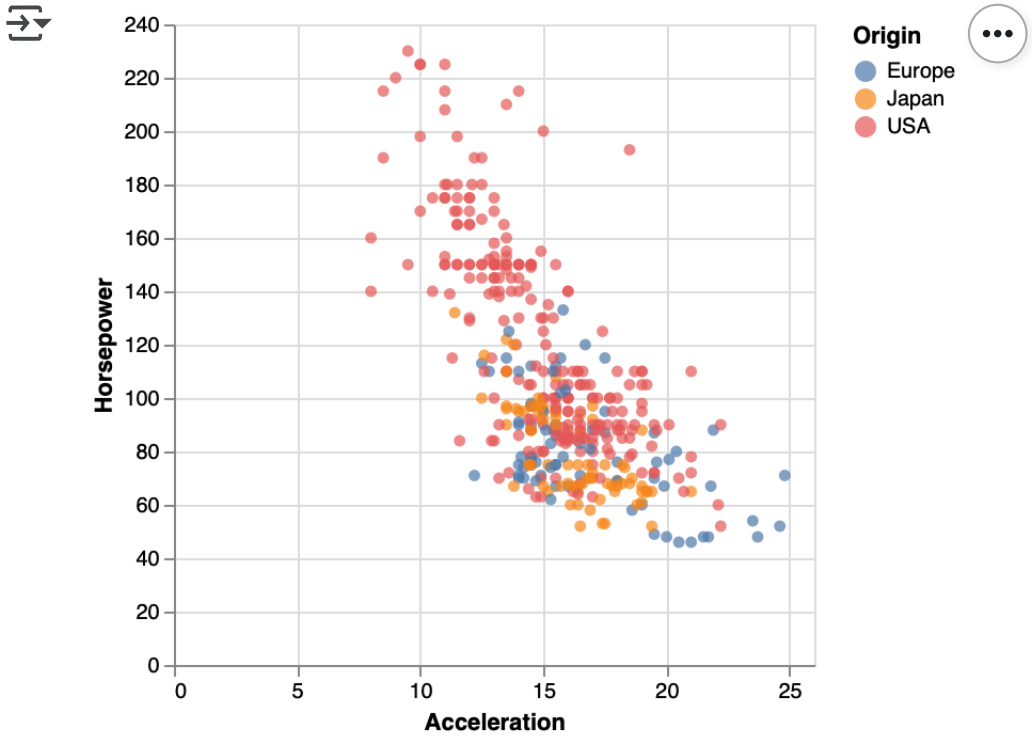


Q: can you create a 2D scatterplot with `Acceleration` and `Horsepower`? Use `Origin` for the colors.

```
# YOUR SOLUTION HERE

import altair as alt

alt.Chart(cars).mark_circle().encode(
    x='Acceleration',
    y='Horsepower',
    color='Origin'
)
```



Because altair/vega-lite/vega are essentially drawing the chart using javascript (and D3.js), it is very easy to export it on the web. Probably the simplest way is just exporting it into an HTML file: https://altair-viz.github.io/getting_started/starting.html#publishing-your-visualization

Save the chart to m07.html and upload it too.

```
# YOUR SOLUTION HERE
chart.save('m07.html')
```